

User Dashboard System: Product + Technical Handoff

Updated: 2026-03-26

Audience: product, design, QA, support, and engineering

Scope: the personal user workspace under `/app/*` with emphasis on `/app/dashboard` (excluding org workspace `/app/org/*` and platform admin `/app/admin/*` except where they influence the user dashboard)

This document is a hybrid of:

- ? technical whitepaper (system model and contracts)
- ? system design doc (components, data flow, security boundaries)
- ? developer handoff guide (setup, reproducibility, extension points)

It is written assuming the reader has zero prior exposure to this repo.

Table of Contents

1. Executive Overview
2. Product Evolution and Iterations
3. System Architecture
4. Technical Stack
5. Detailed Implementation Breakdown
6. Database and Data Modeling
7. User Flow and UX Logic
8. Challenges and Problems Encountered
9. Solutions and Corrections Implemented
10. Performance and Scalability Considerations
11. Deployment and Environment Setup
12. Reproducibility Guide (Critical)
13. Future Improvements

1. Executive Overview

1.1 What the User Dashboard is

The User Dashboard is the landing surface for the personal workspace at `/app/dashboard`.

It is not just a static homepage.

It is a real-time operational summary that:

- ? greets and identifies the signed-in user
- ? shows current KPI counts derived from user-owned rows in the database
- ? provides a launchpad into core workflows (Generate, Calendar, Library, Settings, Help)

- ? shows recent generation activity and account connection health
- ? supports generation search via the top navbar across the user workspace

1.2 Core purpose and value

The dashboard answers: "What is happening with my content today, and what should I do next?"

It reduces cognitive load by:

- ? surfacing the only numbers that matter most in an MVP workflow (generated, scheduled, published, credits)
- ? keeping those numbers live via database subscriptions
- ? giving a user a single place to start work, continue work, or resolve problems (account health, support)

1.3 Target users and use cases

Target users in the personal workspace:

- ? creators and small teams operating in personal mode
- ? admins who also use the product as a normal user (admins can access both workspaces)

Primary use cases:

- ? first-time onboarding (connect account, generate first post)
- ? daily content operations (generate, review recent work, schedule)
- ? lightweight monitoring (scheduled/published counts, account connection status)
- ? support entry (help panel and ticket submission)

2. Product Evolution and Iterations

This section captures the evolution that is visible from in-repo documentation and migrations.

It does not claim to cover conversations that are not stored in this repo.

2.1 Initial idea and problem statement

The MVP goal is to unify content creation and content operations in one workspace:

- ? generate content (prompt -> media -> caption)
- ? turn it into a post lifecycle record
- ? schedule and publish through a calendar
- ? keep a library record for reuse
- ? provide support escalation and admin oversight

2.2 Major iterations (v1 to current)

v1: Personal workspace shell and basic dashboard

Visible in:

- ? src/pages/Dashboard/UserDashboard.jsx
- ? src/components/User/UserNavbar.jsx
- ? src/components/User/UserSidebar.jsx
- ? src/styles/UserDashboard.css

Key characteristics:

- ? dashboard is a summary + launchpad
- ? navigation and styling are shared across user pages by importing UserDashboard.css

v2: Calendar and library alignment (data lifecycle hardening)

Visible in:

- ? supabase/migrations/20260227090000_calendar_library_alignment.sql
- ? supabase/migrations/20260227103000_generation_post_unification_and_rls.sql

Key shift:

- ? The system formalized the contract that completed generations should map to draft posts.
- ? The library layer became a first-class index over posts/media/templates via triggers.

v3: Brand Kit onboarding and storage provisioning

Visible in:

- ? supabase/migrations/20260220041938_brand_kit.sql
- ? supabase/migrations/20260222013000_storage_buckets_and_policies.sql
- ? src/stores/BrandKitStore.js
- ? src/pages/Settings/BrandKitPage.jsx

Key shift:

- ? brand configuration is treated as user-owned state
- ? storage buckets are explicitly provisioned and protected by per-user folder policies

v4: Notifications and help system expansion

Visible in:

- ? supabase/migrations/002_user_notifications.sql
- ? supabase/migrations/20260321153000_admin_v4_notifications_notes_and_activity.sql
- ? supabase/migrations/20260323100000_risk_notifications_and_help_system_core.sql
- ? supabase/migrations/20260323101000_risk_notifications_and_help_system_policies.sql
- ? src/components/User/UserNavbar.jsx (bell)
- ? src/stores/HelpStore.js, src/components/HelpPanel/HelpPanel.jsx, src/pages/HelpPage/HelpPage.jsx

Key shift:

- ? user notifications expanded beyond "admin sent a message" to typed notifications with metadata
- ? help tickets (complaints) were normalized toward a user support center model

v5: Org workspace added (dashboard remains personal)

Visible in:

- ? `src/router/router.jsx` org routes
- ? `src/Context/AuthContext.jsx` workspace redirect resolution

Key shift:

- ? user may now belong to org workspaces as well as personal workspace
- ? dashboard remains the personal default, but routing can redirect into org or context selector based on memberships

2.3 Why changes were made, trade-offs, and rationale

Observed rationale from code/doc choices:

- ? Database triggers were used to make the lifecycle contract enforceable even if multiple UIs write posts/generations.
- ? Realtime subscriptions were used to keep the dashboard and navbar live without manual refresh.
- ? Compatibility-first migrations were favored (add columns, keep legacy columns) to avoid breaking existing environments.

Trade-offs:

- ? More logic lives in the database (triggers, RLS) which improves integrity but increases onboarding burden.
- ? Some core tables appear to predate the committed migration set (reproducibility risk called out later).

3. System Architecture

3.1 High-level architecture (frontend, backend, services)

Frontend:

- ? React pages under `src/pages/`
- ? shared user shell components (`UserNavbar`, `UserSidebar`) imported into each user page
- ? Zustand stores for complex stateful areas (`Generate`, `Calendar`, `Brand Kit`, `Help`)

Backend:

- ? Supabase Auth, Postgres, Realtime, Storage, Edge Functions
- ? RLS defines the security boundary for direct client queries
- ? Triggers define lifecycle glue (generation -> post, post -> library item)

3.2 Workspace topology

In router terms (`src/router/router.jsx`):

- ? Protected app shell: `/app/*` (auth required)
- ? User workspace routes:
 - ? `/app/dashboard`
 - ? `/app/generate` and `/app/generate/:sessionId`
 - ? `/app/calendar`
 - ? `/app/library`
 - ? `/app/help`
 - ? `/app/settings`
 - ? `/app/settings/brand-kit`
- ? Platform admin workspace: `/app/admin/*` (super admin only)
- ? Org workspace: `/app/org/:orgId/*` (org member required)

3.3 Data flow (step-by-step)

The dashboard's core data flow is:

1. Auth resolves user identity and role in `src/Context/AuthContext.jsx`.
2. The dashboard queries user-owned rows:
 - ? profiles for name and credits
 - ? generations for recent activity and counts
 - ? posts for lifecycle counts (draft/scheduled/published)
 - ? `connected_accounts` for account health state
3. The UI renders:
 - ? greeting and CTA
 - ? KPI grid (`RealtimeKPICards` via `useRealtimeKPIs`)
 - ? recent generations list
 - ? quick actions
 - ? account health panel
4. Realtime subscriptions refresh data on change:
 - ? dashboard subscribes to `generations`, `posts`, `connected_accounts`
 - ? KPI hook subscribes to `generations`, `posts`, and `profiles`
 - ? navbar subscribes to `generations`, `posts`, and `user_notifications`

3.4 Key components and interactions

Core dashboard components:

- ? `src/pages/Dashboard/UserDashboard.jsx`
- ? `src/components/Dashboard/RealtimeKPICards.jsx`

? src/hooks/useRealtimeKPIs.js

Shared user shell:

? src/components/User/UserNavbar.jsx

? src/components/User/UserSidebar.jsx

? src/styles/UserDashboard.css (shared theme and shell layout)

Support integration:

? src/components/HelpPanel/HelpPanel.jsx

? src/pages/HelpPage/HelpPage.jsx

? src/stores/HelpStore.js

3.5 State management approach

State is split by responsibility:

? Auth and access resolution: React context (src/Context/AuthContext.jsx)

? Page-local UI state: local useState inside pages (dashboard search query, popovers)

? Complex cross-component workflows: Zustand stores, including:

? useSessionStore for Generate

? useCalendarStore for Calendar

? useBrandKitStore for Brand Kit

? useHelpStore for Help / tickets

The dashboard itself uses local state for:

? loading, greeting, search query

? recent generation data sets

? connected account list

And offloads KPIs to useRealtimeKPIs.

4. Technical Stack

This section lists what the repo actually uses and why (based on code structure and existing docs).

If a decision rationale was not recorded in the repo, it is called out explicitly.

4.1 Frameworks, libraries, and tools used

Frontend:

? React 18

? Vite 7

? React Router 6

- ? Zustand (state stores for complex flows)
- ? React Hot Toast (toasts)
- ? Lucide React (icons)

Backend platform:

- ? Supabase Auth
- ? Supabase Postgres (primary DB)
- ? Supabase Realtime (Postgres changes subscriptions)
- ? Supabase Storage (generated assets, brand assets, complaint screenshots)
- ? Supabase Edge Functions (invites, credits, generation support in other parts of the app)

4.2 Reasons for choosing each technology

Based on observed usage:

- ? React + Router: componentized UI with route-driven workspaces and deep linking.
- ? Vite: fast dev server and simple environment variable model for client config.
- ? Supabase: one platform to cover auth, DB, realtime, storage, and serverless functions.
- ? Zustand: avoids prop drilling in complex, multi-panel workflows (Generate and Calendar).

4.3 Alternatives considered and why rejected

No explicit "alternatives considered" decision log is present in this repo for the user dashboard.

If you need to formalize this, create an ADR set (Architecture Decision Records) and backfill:

- ? why Vite vs Next.js / Remix
- ? why Supabase vs a custom API + DB
- ? why Zustand vs React Query or Redux Toolkit

Until then, only the "observed reasons" above are defensible.

5. Detailed Implementation Breakdown

5.1 Core modules and features of the dashboard

The dashboard is composed of five major modules:

1. App shell layout (navbar + sidebar)
2. Greeting and first-time onboarding checklist
3. KPI grid
4. Recent generation activity list and search
5. Account health and quick actions

5.2 Key files

Dashboard page:

? src/pages/Dashboard/UserDashboard.jsx

KPI subsystem:

? src/components/Dashboard/RealtimeKPICards.jsx

? src/hooks/useRealtimeKPIs.js

User shell:

? src/components/User/UserNavbar.jsx

? src/components/User/UserSidebar.jsx

? src/styles/UserDashboard.css

Auth and routing:

? src/Context/AuthContext.jsx

? src/utils/protectedRoute.jsx

? src/utils/authRouting.js

? src/utils/PostAuthRedirect.jsx

? src/router/router.jsx

Related pages that share the dashboard shell:

? src/pages/GeneratePage/GeneratePageV2.jsx

? src/pages/CalendarPage/CalendarPageV2.jsx

? src/pages/LibraryPage/LibraryPageV2.jsx

? src/pages/Settings.jsx

? src/pages/Settings/BrandKitPage.jsx

? src/pages/HelpPage/HelpPage.jsx

5.3 Critical code paths and logic

5.3.1 Dashboard data fetch and realtime refresh

UserDashboard performs:

? a multi-query fetch to build the summary state

? a realtime subscription that triggers refetch when key tables change

Observed realtime tables:

? generations

? posts

? connected_accounts

Trade-off:

- ? Simple correctness via refetch
- ? Potential extra load if subscriptions are broad or if the user has a large history

5.3.2 KPI definitions and realtime KPI hook

useRealtimeKPIs(userId) defines:

- ? totalGenerated: count of generations excluding failed
- ? scheduledPosts: count of posts with status='scheduled'
- ? published: count of posts with status='published'
- ? creditsLeft: profiles.credits (or compatible field)

The hook subscribes to:

- ? generations changes filtered by user id
- ? posts changes filtered by user id
- ? profiles updates filtered by user id

5.3.3 Navbar search and notifications

UserNavbar provides:

- ? search across the user's generation index by title/prompt
- ? notifications aggregation from:
 - ? generations (status updates)
 - ? posts (scheduled/published/failed)
 - ? user_notifications (admin/system notifications)

Unread semantics:

- ? generation/post items use a local "seen at" timestamp stored in localStorage
- ? user_notifications unread is based on DB is_read

5.3.4 Sidebar navigation and cross-page integration

UserSidebar defines the user workspace nav and persists collapsed state.

It also integrates:

- ? Brand Kit status (configured vs incomplete)
- ? Help ticket unread count for resolved tickets that have not been acknowledged (user_notified_at null)

6. Database and Data Modeling

This section documents the DB model that the user dashboard depends on.

Important note:

- ? Not every "base table create" statement for profiles, sessions, generations, and posts appears in the committed migration set in this repo.
- ? Later migrations assume those tables already exist and then add RLS, triggers, and columns.
- ? This is a known reproducibility risk that is addressed in Section 12.

6.1 Key entities and relationships (conceptual)

Primary identity:

- ? `auth.users` is the identity provider table managed by Supabase Auth.
- ? `profiles.id` is expected to match `auth.users.id` and acts as the app-level user profile record.

Content lifecycle:

- ? `sessions` group work in Generate.
- ? `generations` are the canonical "creation artifact" from AI generation.
- ? `posts` are the canonical "publishing lifecycle artifact" (draft -> scheduled -> publishing -> published/failed).

Account connectivity:

- ? `connected_accounts` represent destinations and connection status.

Library indexing:

- ? `content_library_items` provides a unified library index over:
 - ? `posts`
 - ? uploaded media assets
 - ? content templates

Support:

- ? `complaints` represent user-submitted support tickets.
- ? screenshot assets are stored in a private storage bucket and referenced by `complaints.screenshot_url`.

Notifications:

- ? `user_notifications` represent admin or system notifications to the user (in-app bell).

6.2 Important migrations and contracts

Generation-to-post unification contract

Migration:

- ? `supabase/migrations/20260227103000_generation_post_unification_and_rls.sql`

Contract:

- ? when a generation becomes completed, the DB ensures a draft post exists for it
- ? uniqueness is protected by an index to prevent duplicate draft rows per generation/account pair

This is a core architectural decision:

- ? UI does not need to remember to create a post row
- ? downstream systems can treat posts as the unified lifecycle table

Calendar/library triggers

Migration:

- ? supabase/migrations/20260227090000_calendar_library_alignment.sql

Contracts:

- ? lock_terminal_posts() guards against illegal lifecycle regression from published/publishing
- ? create_library_item_from_post() ensures posts appear in the library index

Brand kit schema and storage

Migrations:

- ? supabase/migrations/20260220041938_brand_kit.sql
- ? supabase/migrations/20260222013000_storage_buckets_and_policies.sql

Contracts:

- ? each user has at most one brand_kit row (unique on user_id)
- ? brand assets are user-scoped by RLS and stored at {user_id}/{asset_type}/... inside the brand_assets bucket

Connected accounts contract

Migration:

- ? supabase/migrations/20260321113000_admin_moderation_schema_alignment.sql

Contracts:

- ? connected_accounts exists, is RLS-protected, and supports user-scoped reads/writes
- ? indexes exist for common user queries and admin moderation queries

Help system and notifications

Migrations:

- ? supabase/migrations/002_user_notifications.sql
- ? supabase/migrations/20260323100000_risk_notifications_and_help_system_core.sql
- ? supabase/migrations/20260323101000_risk_notifications_and_help_system_policies.sql

Contracts:

- ? user_notifications carries typed notifications, metadata, and read tracking
- ? complaints is normalized toward a user help center ticket model

6.3 Data lifecycle (CRUD flows)

Create generation

Creation happens in the Generate workflow, but the dashboard consumes the result:

- ? generations rows are created for the user
- ? when status becomes completed, DB ensures a posts draft exists

Dashboard reads:

- ? recent generations list
- ? KPI counts

Schedule/publish posts

Scheduling happens in Calendar and/or Generate post-production.

Dashboard reads:

- ? scheduled count
- ? published count

Connect accounts

Account connections happen in Settings.

Dashboard reads:

- ? connected account count
- ? connection health status pills

Submit support ticket

Help flow:

- ? upload screenshot to storage (bucket: complaint-screenshots)
- ? insert complaint row in complaints
- ? show it in Help panel and Help page

Read notifications

Navbar bell flow:

- ? fetch generation/post-derived notifications (computed)
- ? fetch user_notifications (stored)
- ? mark user_notifications.is_read when clicked

7. User Flow and UX Logic

This section describes what users actually experience and the state transitions the UI implements.

7.1 Post-login routing and workspace selection

Route guards and redirect behavior:

- ? /app/* is protected by ProtectedRoute in src/Utils/protectedRoute.jsx.
- ? The app resolves an effective role and a "workspace redirect path" in src/Context/AuthContext.jsx.
- ? Default personal home is /app/dashboard (USER_HOME_PATH).

Workspace redirection is based on:

- ? admin role detection (super admin goes to /app/admin)
- ? org memberships (may route into /select-context or an org overview)
- ? last-used context (context_last_used)

Practical user outcomes:

- ? a pure personal user goes to /app/dashboard
- ? a user with org memberships but no last context may be sent to /select-context
- ? an admin is routed into /app/admin by default, but can still visit /app/dashboard

7.2 First-time onboarding checklist

The dashboard uses a simplified onboarding checklist:

- ? create account (always done if logged in)
- ? connect a social account (based on connected_accounts count)
- ? generate a first post (based on generations count)

UX intent:

- ? reduce the "blank dashboard" problem
- ? push the user into the first real value loop quickly

7.3 Generation search UX

Search behavior:

- ? top navbar includes a search box intended for generation history search
- ? Ctrl/Cmd+K focuses the search input

- ? Enter selects the first result

- ? selecting a result routes to the generate workflow with a session deep-link and hash anchor

7.4 Notifications UX

The bell aggregates:

- ? generation status updates

- ? post lifecycle updates (scheduled/published/failed)

- ? stored user_notifications (admin/system messages)

Read behavior:

- ? opening the bell updates a "timeline seen at" timestamp in localStorage for derived notifications

- ? clicking a stored user notification updates user_notifications.is_read and read_at

- ? complaint-resolved notifications route into /app/help?tab=tickets

7.5 Account health UX

Account health is computed from connected_accounts rows:

- ? platform label

- ? account name

- ? connection status pill derived from connection_status

Typical statuses handled:

- ? active/mock

- ? expired

- ? error/failed

- ? revoked/disconnected

7.6 Cross-page shell UX contract

All major user workspace pages use the same shell:

- ? <div className="dashboard-shell">

- ? <UserNavbar />

- ? <UserSidebar />

And share the same core design tokens defined in src/styles/UserDashboard.css.

8. Challenges and Problems Encountered

This section documents challenges that are visible in existing audits and code patterns.

8.1 Status enum drift across user pages

Documented in:

? docs/user-pages-audit-and-improvement-plan.md

Risk:

? different parts of the product historically used posted vs published

? inconsistent status reading causes KPIs, notifications, and admin moderation to disagree

8.2 Realtime fan-out and refetch patterns

Dashboard correctness is currently achieved by refetching on changes.

Risk:

? broad subscriptions and multi-query refetch can cause unnecessary load for large datasets

8.3 Migration completeness

Some core tables are altered and protected in migrations but are not created from scratch in this repo's migration set.

Risk:

? new environments can drift unless the baseline schema is provided

8.4 Compatibility-first evolution

The code and migrations contain "fallback select variants" and compatibility columns.

Benefit:

? safer upgrades across partially migrated environments

Cost:

? higher cognitive load for new developers

9. Solutions and Corrections Implemented

This section summarizes corrective patterns already used in the repo.

9.1 Canonical lifecycle unification (generation -> draft post)

Solution:

? ensure_draft_post_for_generation() trigger ensures the post lifecycle starts consistently

Why it works:

? it makes the lifecycle contract database-enforced rather than UI-enforced

9.2 Library indexing via triggers

Solution:

- ? post insert creates a `content_library_items` record

Why it works:

- ? library browsing stays consistent even if posts are created by different flows

9.3 Storage provisioning and per-user policies

Solution:

- ? explicit bucket provisioning and per-user folder policies

Why it works:

- ? prevents "bucket not found" failures and reduces accidental cross-user access

9.4 Notification normalization

Solution:

- ? compatibility columns and triggers normalize user/admin notification shapes

Why it works:

- ? keeps the UI stable even as notification schemas evolve

10. Performance and Scalability Considerations

10.1 Known bottlenecks

- ? KPI counts use `count ( 'exact' )` across posts and generations
- ? dashboard refetches multiple query sets on every subscription event
- ? notification bell aggregates several sources every time it refreshes

10.2 Optimizations already present

- ? KPI realtime subscriptions are filtered to the current user id
- ? sidebar/help uses simple local derivations (unread counts computed locally)
- ? some heavy loads defer via idle callbacks or delayed prefetch

10.3 Recommended scaling strategy

If this system grows beyond MVP scale, the likely next steps are:

- ? replace multi-query KPI computation with a server-side view or RPC returning a single KPI row
- ? subscribe to fewer tables and update incrementally instead of refetching everything

- ? add pagination and search indexes for generation history instead of fixed limits
- ? use materialized aggregates for dashboard charts when real analytics arrives

11. Deployment and Environment Setup

11.1 Local setup (frontend)

1. Install dependencies

```
npm install
```

2. Add Vite environment variables for Supabase client config

```
VITE_SUPABASE_URL=...  
VITE_SUPABASE_ANON_KEY=...
```

Optional (affects AI-backed flows elsewhere in the workspace):

```
VITE_GROQ_API_KEY=...  
VITE_GROK_API_KEY=...  
VITE_LLM_PROVIDER=...  
VITE_GROQ_VISION_MODEL=...  
VITE_GROK_MODEL=...
```

3. Run dev server

```
npm run dev
```

11.2 Supabase setup (DB, storage, functions)

Apply migrations:

```
supabase db push
```

Storage buckets:

- ? generated_assets (public)
- ? brand_assets (private)

Optional buckets (support / screenshots):

- ? complaint-screenshots (private)

Edge function secrets needed for some flows:

- ? SUPABASE_URL
- ? SUPABASE_SERVICE_ROLE_KEY
- ? APP_URL
- ? optional email provider keys (for invite flows in other workspaces)

11.3 Build and deploy

Build:

```
npm run build
```

Deploy (repo uses GitHub Pages):

```
npm run deploy
```

12. Reproducibility Guide (Critical)

This section is explicit about what you can and cannot reproduce from only the committed migration set.

12.1 The reproducibility constraint

The repo migrations include:

? RLS policies, triggers, and schema extensions for core tables (sessions, generations, posts, profiles)

But do not include the original CREATE TABLE statements for those core tables.

That means:

? a brand-new Supabase project cannot be made identical using only the migrations in this repo unless you also supply the baseline schema for those tables.

This limitation is consistent with the repo's own MVP docs that call out migration governance as in-progress.

12.2 Exact reproduction path (recommended)

To reproduce the current system exactly:

1. Start from the canonical Supabase project/schema that this repo was developed against.
2. Use `supabase migration fetch` to import the missing baseline migrations into `supabase/migrations`.
3. Apply the full migration set in order.

12.3 Practical reproduction steps

1. Create or link a Supabase project.
2. Ensure core tables exist (profiles, sessions, generations, posts).
3. Apply repo migrations:

? 20260220041938_brand_kit.sql

? 20260222013000_storage_buckets_and_policies.sql

? 20260227090000_calendar_library_alignment.sql

? 20260227103000_generation_post_unification_and_rls.sql

? 002_user_notifications.sql

? 20260321113000_admin_moderation_schema_alignment.sql (connected accounts + compatibility columns)

? 20260323100000_risk_notifications_and_help_system_core.sql

? 20260323101000_risk_notifications_and_help_system_policies.sql

? plus any other migrations required by your deployment target

4. Start the frontend with valid VITE_SUPABASE_URL and VITE_SUPABASE_ANON_KEY.

12.4 Checkpoints (verify correctness)

Checkpoint A: routing and auth

- ? sign in and land on /app/dashboard
- ? navbar and sidebar render

Checkpoint B: KPI correctness

- ? create a generation and confirm Total Generated increments
- ? confirm a completed generation leads to a draft post existing (DB trigger contract)

Checkpoint C: notifications

- ? change a generation status and confirm bell updates
- ? insert a user_notifications row and confirm it appears and can be marked read

Checkpoint D: connected accounts and account health

- ? create a connected_accounts row and confirm account health cards render

Checkpoint E: help tickets

- ? submit a complaint and confirm it appears in the help panel and help page

13. Future Improvements

Known limitations and suggested enhancements:

- ? Replace dashboard refetch-on-any-change with incremental updates or a single KPI RPC.
- ? Add a real analytics page instead of routing /app/analytics to the calendar.
- ? Standardize status enum usage everywhere and remove legacy posted references.
- ? Finish migration governance by committing the baseline CREATE TABLE migrations for core tables.
- ? Add integration tests for the core loop: generate -> draft -> schedule -> dashboard KPI update.